

Hello World

Implement the following in a new project, within a file titled HelloWorld.cpp, and then build and execute your program

Looking at the different parts of the program, we have the following:

□

The C++ language defines several headers, which contain information that is either necessary or useful to your program. For this program, the header `<iostream>` is needed.

□

The line using namespace std; tells the compiler to use the std namespace. Namespaces are a relatively recent addition to C++.

□

The next line `// main()` is where program execution begins.' is a single-line comment available in C++. Single-line comments begin with `//` and stop at the end of the line.

□

The line `int main()` is the main function where program execution begins.

□

The next line `cout << "Hello World";` causes the message "Hello World" to be displayed on the screen.

□

The next line `return 0;` terminates `main()` function and causes it to return the value 0 to the calling process.

□

The `cout` object, together with the insertion operator (`<<`), is used to output values/print text

□

You can add as many `cout` objects as you want. However, note that it does not insert a new

line at the end of the output

C++ New Lines

To insert a new line, you can use the `\n` character. Implement the following in a new project, within a file titled `NewLines.cpp`, and then build and execute your program

```
#include <iostream>
using namespace std;
// main() is where program execution begins.
int main() {
    cout << "Hello World"; // prints Hello World
    return 0;
}
```

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello World! \n";
    cout << "I am learning C++";
    return 0;
}
```

2

Two `\n` characters after each other will create a blank line. Modify your `NewLines.cpp` file as follows, and run your program.

Another way to insert a new line, is with the `endl` manipulator.

Modify your `NewLines.cpp` file as follows, and run your program.

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello World! \n\n";
    cout << "I am learning C++";
    return 0;
}

#include <iostream>
using namespace std;
int main() {
    cout << "Hello World!" << endl;
    cout << "I am learning C++";
}
```

```
return 0;  
} c++ lab
```

Alright, let's break this **C++ Lab** down in a way that makes you understand **every single piece** of what is happening. Think of a C++ program like a **movie production** 🎬—every part has a role that allows the program to run correctly.

C++ LAB: Hello World & New Lines

We will understand 3 main things from this lab:

- 1 Writing your first C++ program
- 2 Understanding program structure
- 3 Printing new lines using `\n` and `endl`

1 The Classic First Program: Hello World

Create a file called:

HelloWorld.cpp



Write this program inside it:

```
#include <iostream>  
using namespace std;  
  
// main() is where program execution begins.  
int main() {  
  
    cout << "Hello World";  
}
```

</> C++



Now let's break down **EVERY LINE**.

◆ Line 1: #include <iostream>

⌕ C++



This line tells the compiler:

"Hey C++, please bring the **input/output tools** I need."

What is `iostream` ?

`iostream` means:

Input Output Stream



It allows your program to:

Action	Example
Print text	<code>cout</code>
Get input	<code>cin</code>
Display errors	<code>cerr</code>

Without `iostream`, this will fail:

⌕ C++



Because the compiler wouldn't know what **cout** is.

Think of it like **importing tools from a toolbox** 🧰.

◆ Line 2: using namespace std;

⌕ C++



C++ organizes many things inside something called a **namespace**.

What is a Namespace?

A namespace is like a **folder** 📁 that stores names.

For example:

```
std
├── cout
├── cin
└── endl
```



std means **standard library**.

Without this line you would have to write:

```
<> C++
```



But with:

```
<> C++
```



You can write simply:

```
<> C++
```



Much cleaner 😎.

◆ Comments

```
<> C++
```



This is a **single-line comment**.

Comments are **ignored by the compiler**.

They are only for **humans reading the code**.

Example:

`</> C++`



You use comments to:

- ✓ Explain code
- ✓ Help other programmers
- ✓ Remind yourself what code does

◆ The `main()` Function

`</> C++`



This is the **most important function in C++**.

Every C++ program **starts execution here**.

Think of it like the **main door 🚪** of the program.

Structure:

Program starts



`main()` runs



program executes statements



Why `int` ?

</> C++



`int` means **integer**.

It tells the operating system:

"This program will return an integer value when it finishes."

◆ Curly Braces { }

{

</> C++



These define the **body of the function**.

Everything inside { } belongs to the function.

Example:

```
int main() {  
    cout << "Hello";  
}
```

</> C++



◆ Printing Output: `cout`

</> C++



`cout` means:

character output



It sends text to the **screen (console)**.

The << Operator

<> C++



<< is called the **insertion operator**.

It means:

“Send this data to cout.”

So this line means:

Send "Hello World" to the screen



Output:

Hello World



◆ Ending the Program

<> C++



This tells the operating system:

“The program finished successfully.”

Why 0 ?

In programming:

0 = success
non-zero = error



◆ Full Flow of the Program

When you run the program:

1. Program starts
2. main() runs
3. cout prints "Hello World"
4. program returns 0
5. program ends



Output:

Hello World



2 Printing Multiple Lines

Create another file:

NewLines.cpp



Program:

```
#include <iostream>
using namespace std;

int main() {

    cout << "Hello World! \n";
    cout << "I am learning C++";
```

</> C++



◆ What is `\n`?

`\n` means:

new line



It tells the computer:

"Move the cursor to the next line."

Without `\n`

```
</> C++
```



Output:

```
HelloWorld
```



Everything prints **on the same line**.

With `\n`

```
</> C++
```



Output:

```
Hello  
World
```



3 Creating Blank Lines

If you write:

```
</> C++
```



Output:

```
Hello World!
```



```
I am learning C++
```

Two `\n` create **one blank line**.

4 Using `endl`

Another method:

```
</> C++
```



Output:

```
Hello World!
```



```
I am learning C++
```

◆ What is `endl` ?

`endl` means:

```
end line
```



It:

- 1 Moves to a new line
- 2 Flushes the output buffer

◆ Difference Between `\n` and `endl`

Feature	<code>\n</code>	<code>endl</code>
Creates new line	✓	✓
Flushes buffer	✗	✓
Faster	✓	✗

Programmers often prefer:

`\n`



because it is **faster**.

◆ Multiple `cout`

You can use many `cout` statements:

```
cout << "C++ ";
```

⌄ C++



Output:

C++ is awesome!



🧠 Final Clean Example

```
#include <iostream>
using namespace std;

int main() {
```



```
cout << "Hello World!" << endl;  
cout << "Welcome to C++ programming\n";  
cout << "This is your first lab.";
```

</> C++



Output:

```
Hello World!  
Welcome to C++ programming  
This is your first lab.
```



What Your Lecturer Wants You To Do

You must:

1 Create HelloWorld.cpp

Hello World



2 Create NewLines.cpp

Test:

- \n
- \n\n
- endl

Compile and run.



Pro Tip (Important for Exams)

Remember these definitions:

`iostream`

Library used for input and output operations

`cout`

Object used to **display output**

<<

Insertion operator used to **send data to output**

\n

Escape character used to **create a new line**

endl

Manipulator used to **insert a new line and flush the output buffer**

✅ If you want, I can also show you:

- How C++ programs compile step-by-step (compiler → object file → executable)
- How `cout` actually works inside memory
- Why C++ needs semicolons `;` and what happens if you forget them

These things make C++ suddenly make MUCH more sense. 🚀



but i want git to be saving these projects so that at some point i must push it to git hub

Great thinking. Using **Git from the beginning** is exactly how professional developers manage their projects. Let's walk through the **complete workflow** so your C++ labs are saved locally and can later be pushed to **GitHub**. 🚀

What You Want to Achieve

You want this workflow:

Write C++ Code



Save project with Git



Track changes



Push project to GitHub

So every lab like:

HelloWorld.cpp

NewLines.cpp



is safely stored and version-controlled.

1 Install Git (If Not Installed)

First check if Git exists.

Open **Terminal / Command Prompt** and run:

```
</> Bash
```



If installed you will see something like:

```
git version 2.44.0
```



If not installed, download it from:

- <https://git-scm.com> ↗

Install with default settings.

2 Configure Git (One Time Only)

Git needs to know **who you are**.

Run:

```
</> Bash
```



Example:

```
</> Bash
```



This is what GitHub will show for commits.

3 Create Your C++ Lab Folder

Create a folder for all your labs.

Example:

```
CPP-Labs
```



Inside it:

```
CPP-Labs
|
|-- HelloWorld.cpp
|-- NewLines.cpp
```



4 Initialize Git

Go inside the folder in terminal.

Example:

```
</> Bash
```



Then initialize Git:

```
</> Bash
```



You will see:

```
Initialized empty Git repository
```



Git creates a hidden folder:

```
.git
```



This folder tracks all versions of your project.

5 Check Project Status

Run:

```
</> Bash
```



You may see:

```
Untracked files:  
HelloWorld.cpp  
NewLines.cpp
```



This means Git **sees the files but is not tracking them yet.**

6 Add Files to Git

Stage the files:

```
</> Bash
```



The `.` means **add everything in this folder.**

Now check status again:

```
</> Bash
```



 **Dash**

You will see:

Changes to be committed



7 Make Your First Commit

Now save the snapshot.

 **Bash**

Now Git permanently records this version.

Think of a commit like:

 Taking a photo of your project

8 Create a GitHub Repository

Go to:

<https://github.com>



Click:

New Repository



Example:

Repository Name: cpp-labs



Do **NOT** initialize with README because you already have files.

Create repository.

9 Connect Your Local Project to GitHub

GitHub will show something like this.

Run inside your project:

```
</> Bash
```



Example:

```
</> Bash
```



10 Push Your Code

Now send your project to GitHub:

```
</> Bash
```



If branch name is `master` :

```
</> Bash
```



Now your project is online.

Your Repository Will Look Like This

```
cpp-labs
|
|-- HelloWorld.cpp
|-- NewLines.cpp
```



Normal Daily Workflow

Whenever you add new lab files:

Example:

```
Loops.cpp
Functions.cpp
Arrays.cpp
```



You do:

```
git add .
git commit -m "Added loops lab"
git push
```



Simple.

Professional Structure (Recommended)

Instead of putting everything in one folder, organize it.

```
cpp-labs
|
|-- Lab01
|   |-- HelloWorld.cpp
|
|-- Lab02
|   |-- NewLines.cpp
|
|-- Lab03
|   |-- Variables.cpp
```



Much cleaner.

Bonus: Add a .gitignore

This prevents unnecessary files from going to GitHub.

Create file:


```
.gitignore
```



Example content:

```
*.exe  
*.o  
*.out  
*.log
```



This avoids uploading compiled files.

Example Final GitHub Project

```
cpp-labs  
|  
|-- Lab01  
|    |-- HelloWorld.cpp  
|  
|-- Lab02  
|    |-- NewLines.cpp  
|  
|-- .gitignore  
|  
|-- README.md
```



This looks **very professional**.

If you want, I can also show you:

- How professionals structure C++ projects with Git
- The 7 Git commands every programmer must know
- How to never lose code again using Git branches
- How to use Git directly inside VS Code

Just say “teach me the professional Git workflow” and I’ll show you something most beginners learn years later. 😎



Data Types and Variables

While writing program in any language, you need to use various variables to store various information. Variables are nothing but reserved memory locations to store values. This means that when you create a variable you reserve some space in memory.

You may like to store information of various data types like character, wide character, integer, floating point, double floating point, boolean etc. Based on the data type of a variable, the operating system allocates memory and decides what can be stored in the reserved memory.

Primitive Built-in Types

C++ offers the programmer a rich assortment of built-in as well as user defined data types.

Following table lists down seven basic C++ data types.

Type

Keyword

Boolean

bool

Character

char

Integer

int

Floating point

float

Double floating point

double

Valueless

void

Wide character

wchar_t

Several of the basic types can be modified using one or more of these type modifiers:

□ signed

□ unsigned

□ short

□ long

2

Implement the following in a new project, within a file titled DataTypes.cpp, and then build and execute your program.

The sizeof() operator to get size of various data types.

Local Variables

Variables that are declared inside a function or block are local variables. They can be used only by statements that are inside that function or block of code. Local variables are not known to functions outside their own.

Implement the following in a new project, within a file titled LocalVariables.cpp and execute your program.

```
#include <iostream>
using namespace std;
int main() {
    cout << "Size of char : " << sizeof(char) << endl;
    cout << "Size of int : " << sizeof(int) << endl;
    cout << "Size of short int : " << sizeof(short int) << endl;
    cout << "Size of long int : " << sizeof(long int) << endl;
    cout << "Size of float : " << sizeof(float) << endl;
    cout << "Size of double : " << sizeof(double) << endl;
    cout << "Size of wchar_t : " << sizeof(wchar_t) << endl;
    return 0;
}
```

Let's break this **C++ Lab on Data Types and Variables** down in a **clear, deep, beginner-friendly way** so you understand **what every word means and why it exists**. Think of this as understanding **how memory works when you write a C++ program**. 🧠💻

C++ Lab: Data Types and Variables

In this lab you learn three big ideas:

- 1 Variables
- 2 Data types

3 How much memory each type uses



1 What Is a Variable?

A **variable** is simply a **named storage location in memory**.

Imagine your computer's memory like a **huge warehouse with storage boxes** 📦.

Each box stores a value.

Example:

```
</> C++
```



Here:

Part	Meaning
int	data type
age	variable name
20	stored value

Memory view:

Variable Name: age
Value: 20
Memory location: reserved by the program



So a variable is basically:

Name → Memory Location → Stored Value



Why Variables Need Data Types

The computer must know **what type of data you want to store** so it can allocate the correct memory size.

For example:

Data	Type
Letter	char
Whole number	int
Decimal number	float
True/False	bool

If you store the wrong type, the program may fail.

Example:

'A' → char
25 → int
3.14 → float

 2

Primitive Built-in Data Types

C++ already provides **basic data types** built into the language.

These are called **primitive types**.

The 7 main C++ data types

Type	Keyword	Example
Boolean	bool	true / false
Character	char	'A'
Integer	int	10
Floating point	float	3.14
Double floating point	double	3.141592
Valueless	void	no value

Type	Keyword	Example
Wide character	wchar_t	extended characters

◆ bool (Boolean)

Stores only **two values**:

```
true  
false
```



Example:

```
</> C++
```



Memory:

```
1 byte usually
```



Used for:

- ✓ conditions
- ✓ comparisons
- ✓ logic

◆ char (Character)

Stores **one character**.

Example:

```
</> C++
```



Important:

Characters use single quotes



Correct:

'A'



Wrong:

"A"



Memory:

1 byte



◆ int (Integer)

Stores **whole** numbers.

Example:

```
</> C++
```



Memory usually:

4 bytes



Range approximately:

-2,147,483,648 to 2,147,483,647



◆ float (Floating Point)

Stores **decimal** numbers.

Example:

 C++

Memory:

4 bytes



Precision:

~6 decimal digits



◆ double

Stores **large decimal numbers with more precision.**

Example:

 C++

Memory:

8 bytes



Precision:

~15 decimal digits



This is why programmers often prefer **double instead of float.**

◆ void

void means:

no value



Example:

 C++

It means the function **does not return anything**.

◆ wchar_t (Wide Character)

Used for **extended characters and international languages**.

Example:

 C++

Memory:

2 or 4 bytes depending on system



Used for:

- Unicode
 - international characters
-

⚙️ Type Modifiers

Some types can be **modified**.

These modifiers change:

size
range
behavior



Modifiers:

signed
unsigned



short
long

◆ signed

Allows **negative and positive numbers**.

Example:

⟨⟩ C++



Range:

-100 to +100 (example)



◆ unsigned

Only **positive numbers**.

Example:

⟨⟩ C++



Range:

0 to positive numbers



Why useful?

Because removing negatives **doubles the positive range**.

◆ short

Uses **smaller memory**.

Example:

```
</> C++
```



Memory usually:

2 bytes



◆ long

Uses larger memory.

Example:

```
</> C++
```



Memory:

8 bytes (system dependent)



Lab 1: Checking Data Type Sizes

Create file:

DataTypes.cpp



Program:

```
#include <iostream>
using namespace std;

int main() {

    cout << "Size of char : " << sizeof(char) << endl;
```

```
cout << "Size of int : " << sizeof(int) << endl;
cout << "Size of short int : " << sizeof(short int) << endl;
cout << "Size of long int : " << sizeof(long int) << endl;
cout << "Size of float : " << sizeof(float) << endl;
cout << "Size of double : " << sizeof(double) << endl;
cout << "Size of wchar_t : " << sizeof(wchar_t) << endl;

return 0;
```

</> C++



Understanding sizeof()

sizeof() is an **operator** that returns the **memory size of a data type**.

Example:

</> C++



This asks:

How many bytes does an int use?



Example result:

Size of int : 4



Meaning:

int uses 4 bytes



What Is a Byte?

A **byte** is a unit of memory.

1 byte = 8 bits



Example:

```
char → 1 byte  
int → 4 bytes  
double → 8 bytes
```



Example Output

Running the program might produce:

```
Size of char : 1  
Size of int : 4  
Size of short int : 2  
Size of long int : 8  
Size of float : 4  
Size of double : 8  
Size of wchar_t : 4
```



Note:

This can **change depending on the system.**

3 Local Variables

Local variables are variables declared **inside a function or block.**

Example:

```
int main() {  
  
    int x = 10;  
  
}
```

 C++



Here:

x



is **local to main()**.

This means:

Only `main()` can use it



Other functions **cannot see it**.

Example

```
#include <iostream>
using namespace std;

int main() {

    int age = 20;

    cout << age;

    return 0;

} // C++
```



Here:

age



exists **only inside main()**.

When the function finishes:

the variable disappears from memory



Visual Memory Idea

When program starts:

```
main()
|
|-- age = 20
|-- price = 15
```



When `main()` ends:

all local variables are destroyed



Why Local Variables Are Important

Local variables:

- ✓ save memory
- ✓ avoid conflicts
- ✓ keep programs organized

Example:

Two functions can use the same variable name.

```
functionA() → int x
functionB() → int x
```



They **do not** interfere.

What Your Lab Requires

Create two files.

File 1

`DataTypes.cpp`



Print **sizes of data types** using `sizeof()` .

File 2

LocalVariables.cpp



Demonstrate variables declared inside main().

Final Big Picture

When you write:

```
</> C++
```



This happens:

1. Compiler reads "int"
2. Allocates 4 bytes in memory
3. Labels memory as "age"
4. Stores value 25



Memory representation:

```
Variable: age  
Type: int  
Size: 4 bytes  
Value: 25
```



✅ If you want, I can also explain something that confuses most beginners:

- Where variables are stored in memory (Stack vs Heap)
- Why `int` is 4 bytes and `double` is 8 bytes
- How the CPU actually reads these variables

Once you understand that, C++ suddenly becomes 10× clearer. 🚀



Global Variables

Global variables are defined outside of all the functions.

The global variables will hold their value throughout the life-time of your program. A global variable can be accessed by any function. That is, a global variable is available for use throughout your entire program after its declaration.

Implement the following in a new project, within a file titled GlobalVariables.cpp, and then build and execute your program

```
#include <iostream>
using namespace std;
// Global variable declaration:
int g;
int main () {
// Local variable declaration:
int a, b;
// actual initialization
a = 10;
b = 20;
g = a + b;
cout << g;
return 0;
}
4
```

A program can have same name for local and global variables but value of local variable inside a function will take preference.

Modify you GlobalVariables.cpp file as follows, and then build and execute your program.

Constants

You can use const prefix to declare constants with a specific type.

Implement the following in a new project, within a file Constants.cpp and and then build and execute your program.

```
#include <iostream>
using namespace std;
// Global variable declaration:
int g = 20;
int main () {
```

```
// Local variable declaration:  
int g = 10;  
cout << g;  
return 0;  
}
```

Let's break this **Global Variables + Constants** lab in a way that makes it feel like you're learning how memory and scope work inside C++ 🧠💻

1 What is a Global Variable?

A **global variable** is a variable declared **outside all functions**.

Example:

⌕ C++



This is called global because:

It lives outside main()
It is visible everywhere in the program



Think of it like this:

Imagine your program is a building 🏢

Type	Analogy
Global variable	A notice board in the lobby (everyone can see it)
Local variable	A note inside a specific room (only that room can see it)

Global Variable Rules

- ✓ Declared outside functions
- ✓ Exists for the **entire program lifetime**
- ✓ Can be used in **any function**
- ✓ Keeps its value until program ends

Example Program: Global Variables

Create file:

GlobalVariables.cpp



Code:

```
#include <iostream>
using namespace std;

// Global variable declaration:
int g;

int main() {

    // Local variables
    int a, b;

    a = 10;
    b = 20;

    g = a + b;

    cout << g;
```

</> C++



Step-by-step Execution

Step 1: Program starts

```
main() runs
```



Step 2: Local variables created

```
a = 10  
b = 20
```



Step 3: Global variable used

```
g = a + b = 30
```



Step 4: Output

```
30
```



Key Idea

```
</> C++
```



Even though `g` is outside `main()`, it can still be used inside `main()`.

Because:

```
Global variables are visible everywhere
```



IMPORTANT: Scope Conflict (Very Important Concept)

C++ allows:

```
Same variable name globally and locally
```



But local wins inside the function.



Modified Example (Scope Priority)

Create file:

Constants.cpp



Code:

```
#include <iostream>
using namespace std;

// Global variable
int g = 20;

int main() {

    // Local variable (same name!)
    int g = 10;

    cout << g;
```

</> C++



Output:

10



WHY did it print 10 and not 20?

Because of **variable shadowing**.



What is Shadowing?

When a **local variable** has the same name as a **global variable**, the local one hides the global one inside that function.

Like this:

```
Global g = 20    (hidden)
Local g = 10     (used instead)
```



Rule:

```
Local variable > Global variable (inside function)
```



Visual Memory View

GLOBAL MEMORY:

```
g = 20
```

FUNCTION main():

```
g = 10 ← used here
```

```
cout prints local g
```



Why Global Variables Are Useful

- ✓ Share data across functions
- ✓ Avoid passing parameters repeatedly
- ✓ Useful for constants or shared settings

BUT (Very Important!)

Global variables can be dangerous:

- ✗ Hard to track changes
- ✗ Can be modified anywhere
- ✗ Can cause bugs in big programs



Good Practice (Professional Rule)

Use global variables **ONLY** when necessary.

Most of the time, prefer:

- ✓ Local variables
- ✓ Function parameters



2 Constants in C++

Now let's move to **constants**.



What is a Constant?

A constant is a variable whose value:

CANNOT be changed after initialization



Example Concept

```
</> C++
```



Now:

x = 20 ❌ NOT allowed



Why use constants?

- ✓ Prevent accidental changes
- ✓ Improve safety
- ✓ Make code clearer



Constant Example Program

```
#include <iostream>
using namespace std;

int main() {

    const int g = 10;

    cout << g;
```

</> C++



Output:

10



If you try this:

</> C++



You will get an error ❌

Because:

Constants cannot be modified



Real-Life Analogy

Concept	Example
Variable	Your phone password (can be changed)

Concept**Example**

Constant

Speed limit on a road 🚦 (fixed rule)

 Full Summary **Global Variables**

- Declared outside functions
- Accessible everywhere
- Last for entire program

 Local Variables

- Declared inside functions
- Only usable inside that function
- Destroyed after function ends

 Conflict Rule

Local variable overrides global variable

 **Constants**

- Cannot change value
- Declared using `const`
- Used for safety and fixed values

 If you want next level understanding, I can show you:

- Stack vs Heap memory (THIS explains variables deeply)
- Why global variables are dangerous in real systems
- How professional C++ programmers avoid globals completely

- How memory actually stores `int g = 10`

Just say “**explain memory deeply**” and I’ll take you into real systems-level understanding.



User Input

`cin` is a predefined object that reads data from the keyboard with the extraction operator (`>>`).

The `cin` object is said to be attached to the standard input device, which usually is the keyboard.

Implement the following in a new project, within a file titled `UserInput.cpp`, and then build and execute your program.

Modify your `UserInput.cpp` as follows, and then build and execute your program.

```
#include <iostream>
using namespace std;
int main() {
    int x;
    cout << "Type a number: "; // Type a number and press
    enter
    cin >> x; // Get user input from the keyboard
    cout << "Your number is: " << x;
    return 0;
}

#include <iostream>
using namespace std;
int main() {
    int x, y;
    int sum;
    cout << "Type a number: ";
    cin >> x;
    cout << "Type another number: ";
    cin >> y;
    sum = x + y;
    cout << "Sum is: " << sum;
```

```
return 0;  
}
```

Let's break this **C++ User Input Lab** carefully so you understand **how a program communicates with a user through the keyboard**. This is one of the most important basics in programming because programs are rarely useful if they can't **receive data from users**. 🖥️💻

C++ Lab: User Input with `cin`

In this lab you will learn:

- 1 How programs ask the user for data
 - 2 How the keyboard input enters the program
 - 3 How to store that input in variables
 - 4 How to use that input for calculations
-

1 What is `cin`?

`cin` is a **predefined object in C++** used to read input from the keyboard.

The name means:

`cin` = character input



So when you write:

`</> C++`



You are telling the program:

Read data from the keyboard
and store it inside variable `x`





Standard Input (Important Concept)

`cin` is connected to something called **standard input**.

In most programs:

Standard Input = Keyboard
Standard Output = Screen



So the flow looks like this:

```
User types number
      ↓
Keyboard sends data
      ↓
cin reads data
      ↓
Value stored in variable
```



Program 1: Reading One Number

Create file:

UserInput.cpp



Program:

```
#include <iostream>
using namespace std;

int main() {

    int x;

    cout << "Type a number: ";

    cin >> x;

    cout << "Your number is: " << x;
```



<> C++

}



Understanding Every Line

Include library

<> C++



This allows use of:

```
cin  
cout  
endl
```



Namespace

<> C++



Allows writing:

```
cout  
cin
```



instead of:

```
std::cout  
std::cin
```



Start of program

```
</> C++
```



The program begins execution here.

Declaring variable

```
</> C++
```



This creates a variable named **x**.

Memory view:

```
Variable: x  
Type: int  
Purpose: store user input
```



Display message

```
</> C++
```



This prints a message to guide the user.

Output:

```
Type a number:
```



Reading input

```
</> C++
```



Here two things happen.

- 1 Program waits for input
- 2 User types value
- 3 Value stored inside `x`

Example:

User types:

25



Memory becomes:

`x = 25`



Display result

```
</> C++
```



Output:

Your number is: 25



Execution Example

Program run:

Type a number: 15
Your number is: 15



Flow:

```
User enters 15  
↓  
cin stores 15 in x
```





cout prints x



Program 2: Adding Two Numbers

Now modify the program.

```
#include <iostream>
using namespace std;

int main() {

    int x, y;
    int sum;

    cout << "Type a number: ";
    cin >> x;

    cout << "Type another number: ";
    cin >> y;

    sum = x + y;

    cout << "Sum is: " << sum;
```

</> C++



Breaking It Down

Declaring variables

</> C++



Variables created:


```
x → first number  
y → second number  
sum → result
```



Memory representation:

```
x = ?  
y = ?  
sum = ?
```



First input

```
</> C++
```



User types:

```
10
```



Memory:

```
x = 10
```



Second input

```
</> C++
```



User types:

```
20
```



Memory:

```
y = 20
```



Calculate result

`</> C++`



Computation:

```
sum = 10 + 20  
sum = 30
```



Memory:

```
sum = 30
```



Print result

`</> C++`



Output:

```
Sum is: 30
```



Full Execution Example

```
Type a number: 5  
Type another number: 8  
Sum is: 13
```



What is >> Operator?

The symbol:

>>



is called the **extraction operator**.

Meaning:

Take data from input
and extract it into a variable



Example:

```
</> C++
```



means:

read keyboard input → store in x



Multiple Inputs in One Line

You can even do:

```
</> C++
```



Example input:

5 10



Memory:

```
x = 5  
y = 10
```



Important Beginner Mistake

Wrong:

```
</> C++
```



Correct:

```
</> C++
```



Why?

```
<<  → output  
>>  → input
```



What Your Lab Wants You To Practice

Create `UserInput.cpp`.

Test:

- 1 Reading one number
- 2 Reading two numbers
- 3 Adding them together



Concept Summary

Concept	Meaning
<code>cin</code>	reads input from keyboard
<code>>></code>	extraction operator
<code>cout</code>	prints output
variable	stores user data

 If you want, I can also show you something that helps beginners a lot:

The 5 most common `cin` mistakes students make in labs (and how to avoid them).

And also:

How to read text (strings) like:

Enter your name: John
Hello John



Which is the next step after numbers in C++ input.



The picture I've sent there are three slides let's decode very very well one by one let's start with the first one

Great — let's decode the **FIRST** slide only very deeply. I'll explain every bullet and the code so the idea of **C++ references** becomes crystal clear. 🧠💻

📌 Slide 1 — References in C++



What Are References In C++?

Variable u → **int u=10;**

's' is the Reference Variable to u → **int &s = u;**

Reference Operator

Reference in C++

```
int a = 10;
int &b = a;
```

2000 (Address)

'a', 'b' having same address now

SCALER Topics

Reference &

```
int x = 5;
int &ref = x;
```

X — c200 — 5 — ref

Pointer *

```
int x = 5;
int* ptr = x;
```

X — c200 — 5

This slide introduces one of the **most important concepts** in advanced C++:

References



References are heavily used in:

- functions
- object-oriented programming
- operator overloading
- performance optimization

🧠 1. "A reference is an alias"

The slide says:

A reference is an alias.



Alias means:

Another name for the same thing



Example in real life:

Name	Meaning
Samuel	original name
Sam	alias

Both refer to **the same person**.

In C++ the same idea applies

Example:

```
</> C++
```



Here:

```
exampleVariable  
exampleReference
```



are **two names for the same memory location**.

Memory Visualization

Imagine memory like this:

```
Memory Address: 1000  
Value: 25
```



Normal variable:

```
exampleVariable → address 1000
```



Reference:

```
exampleReference → address 1000
```



Both point to **exactly the same memory**.

2. "A reference is initialized with the name of another object"

The slide says:

A reference is initialized with the name of another object



Example:

```
</> C++
```



Here:

```
x → original variable  
ref → reference (alias)
```



So:

```
ref is another name for x
```



Key Rule

References **must be initialized immediately**.

Correct:

```
</> C++
```



Wrong:

 C++

This causes a **compile error**.

Because a reference must always **refer to something**.

3. "Whatever you do to the reference also affects the target"

The slide says:

Whatever you do to the reference also affects the target



Example:

```
int x = 25;  
int &ref = x;
```

 C++

What happens?

x becomes 56



Because:

ref and x are the SAME memory



Memory Diagram

Before change:

x → 25
ref → 25



After:

```
ref = 56
```



Now:

```
x → 56  
ref → 56
```



Both changed.

4. "References are created like other variables"

The slide says:

```
Declared with a type and a name
```



Example:

```
</> C++
```



Breakdown:

Part	Meaning
int	data type
&	reference operator
ref	variable name

Important Clarification

The slide says:

```
Same symbol as address operator but NOT the same operators
```



This confuses many students.

The symbol:

&



has two meanings in C++.

Meaning 1 — Reference declaration

Example:

`</> C++`



Here:

& means reference



Meaning 2 — Address operator

Example:

`</> C++`



Here:

& means "get memory address"



So the symbol is the same but the meaning depends on context.



5. "References must be initialized when declared"

This is one of the most important rules.

Correct:

`</> C++`



Wrong:

`</> C++`



Why?

Because a reference **must** point to an existing object.

A reference **cannot** exist alone.

Let's Decode the Code on the Slide

The slide shows something like this:

```
int main() {  
  
    int exampleVariable = 25;  
  
    cout << "The value of exampleVariable before ref: "  
        << exampleVariable << endl;  
  
    int &exampleReference = exampleVariable;  
  
    exampleReference = 56;  
  
    cout << "The value of exampleVariable after ref: "  
        << exampleVariable << endl;  
}
```

`</> C++`



Step-by-Step Execution

Step 1

```
exampleVariable = 25
```



Memory:

```
exampleVariable → 25
```



Step 2

```
int &exampleReference = exampleVariable;
```



Now:

```
exampleReference → alias of exampleVariable
```



Memory:

```
exampleVariable → 25  
exampleReference → 25
```



Step 3

```
exampleReference = 56
```



Because reference = variable:

```
exampleVariable becomes 56
```



Step 4 Output

```
The value of exampleVariable before ref: 25  
The value of exampleVariable after ref: 56
```



Key Insight

Reference does **not** create a new variable.

It simply creates:

```
another name  
for the same variable
```



References vs Normal Variables

Example:

≡ > ChatGPT ▾ 📺 Free offer ×



```
</> C++
```



Memory:

```
x → 10  
y → 10
```



Two separate variables.

But reference:

```
</> C++
```



Memory:

```
x → 10  
y → same memory as x
```



Only **one value** exists.

Why References Are Powerful

References are used for:

1 Passing variables to functions efficiently

Example:

```
swap numbers  
modify objects
```



2 Avoid copying large objects

Example:

```
big arrays  
large structures
```



3 Making code cleaner than pointers

References are **safer** than pointers.

Final Mental Model

Think of references like **nicknames**.

```
Alexander → Alex
```



Both names refer to **the same person**.

In C++:

```
exampleVariable  
exampleReference
```



both refer to **the same memory location**.

✅ In the **next slide** things get more interesting because it explains:

```
Addresses of references  
Why references cannot be reassigned  
Why references cannot be null
```



These are **very important rules in advanced C++**.

So next we'll **decode slide 2 extremely carefully**.

